

Chapter 12: Signals

1. Overview

For most of its life, Angular's change detection worked by re-checking entire component trees whenever "something might have changed" — a click, an HTTP response, a timer tick. Zone.js patched every async browser API so Angular would know when to run this check. It worked, but it was coarse-grained: Angular could not tell *which* piece of data changed, only that *some* async task had completed somewhere. The result was a change detection pass that walked far more of the tree than necessary, plus a permanent dependency on Zone.js patching global APIs — a source of bundle weight, debugging pain, and incompatibility with some third-party code.

Signals, introduced as developer preview in Angular 16 and stabilized in Angular 17, are Angular's answer: a **fine-grained, push-based reactivity primitive**. A signal is a wrapper around a value that knows *who read it* and can notify *exactly those consumers* when the value changes. Instead of asking "did anything change?" across the whole tree, Angular can ask "which signals changed, and which computations/templates depend on them?" This is the same class of reactivity model used by SolidJS, Vue 3 (refs), and Preact Signals — dependency-tracked, glitch-free, synchronous-read reactive graphs.

Why this matters architecturally:

- **Fine-grained updates:** only components/templates that actually read a changed signal need to be checked, not the whole tree.
- **Zoneless Angular:** signals give Angular a native notification mechanism for "the UI might need updating," removing the need for Zone.js entirely (`provideZonelessChangeDetection()`).
- **Synchronous, glitch-free reads:** reading a signal always gives you the current, consistent value — no stale intermediate states during a batch of updates, unlike naive observable chains where intermediate values can be observed.
- **Simpler local state:** signals give component-local reactive state without needing RxJS subscriptions, `async` pipes, or manual unsubscription for the common "just hold a value and react to it" case.
- **A unified model for component APIs:** `input()`, `model()`, `output()`, and `viewChild()` / `contentChild()` signal-based queries let component authoring be expressed entirely in the signal primitive, replacing `@Input` / `@Output` / decorator-based queries with plain functions.

Signals are not a replacement for RxJS — they solve a different problem (synchronous reactive *state*, not asynchronous *event streams*). Volume 3 covers the comparison and interop (`toSignal` / `toObservable`) in depth. This chapter is scoped to the signal primitives themselves.

2. Core Concepts

2.1 `signal()` — writable signals

`signal(initialValue)` creates a **writable signal**: a getter function that also carries `.set()` and `.update()` methods.

```
import { signal } from '@angular/core';

const count = signal(0);

count();           // read – call it like a function → 0
count.set(5);      // overwrite
count.update(v => v + 1); // derive next value from current value
```

Key properties:

- Reading is done by **invoking the signal as a function**: `count()`, not `count`. This is what lets Angular do automatic dependency tracking — every read is an explicit function call that can be intercepted.
- Signals hold a **synchronous, current value** — there's no subscription needed to get the "latest" value; you just call it.
- Equality checking: by default `signal()` uses `Object.is` to decide whether a new value actually differs from the old one. If `Object.is(oldValue, newValue)` is true, no change is recorded and dependents are not notified. You can override this with a custom equality function:

```
const user = signal(initialUser, {
  equal: (a, b) => a.id === b.id,
});
```

This matters for object/array values — pushing a new array reference with the same *contents* will still trigger updates unless you supply a custom `equal`.

2.2 `computed()` — derived, read-only, memoized signals

`computed()` creates a signal whose value is derived from other signals via a pure function. It is **read-only** (no `.set()` / `.update()`) and **lazily evaluated with memoization**.

```
import { signal, computed } from '@angular/core';

const price = signal(100);
const quantity = signal(2);
const total = computed(() => price() * quantity());

total(); // 200
price.set(150);
total(); // 300 – recomputed only because a dependency changed
```

Characteristics:

- **Automatic dependency tracking**: `computed()` doesn't need you to declare dependencies — it records every signal read *during* the last execution of its function and subscribes to exactly those.
- **Lazy**: the derivation function doesn't run at creation time; it runs on first read, and its result is cached until a dependency signal changes.
- **Memoized**: repeated reads without an intervening dependency change return the cached value without re-running the function — so put pure, cheap-to-run (or cached-if-expensive) logic in there, not side effects.

- Dependencies are **dynamic**: if a `computed()` conditionally reads different signals on different runs (e.g., an `if` branch), the tracked dependency set updates to match the branch actually taken on the most recent run.

2.3 `effect()` — side effects that react to signal changes

`effect()` registers a function that Angular **automatically re-runs whenever any signal it reads changes**, for the purpose of running **side effects** (logging, syncing to `localStorage`, manual DOM work, calling third-party non-Angular APIs) — not for computing values (use `computed()` for that).

```
import { Component, signal, effect } from '@angular/core';

@Component({ ... })
export class SearchComponent {
  query = signal('');

  constructor() {
    effect(() => {
      console.log('Query changed to:', this.query());
      localStorage.setItem('lastQuery', this.query());
    });
  }
}
```

Rules that matter for interviews:

- By default, `effect()` must be created in an **injection context** (constructor, field initializer, or with an explicit `injector` option) — it needs to tie its lifecycle to something that can clean it up.
- Effects run **asynchronously**, after the current change-detection cycle, batched — not synchronously the instant a signal changes.
- Effects run **at least once** to establish their dependencies, even before any signal changes.
- An effect **automatically cleans up** (stops running) when its owning context (e.g., the component) is destroyed. You can also register a manual cleanup via the `onCleanup` callback passed into the effect function, for things like clearing timers between runs.
- Writing to a signal *inside* the same `effect()` that reads it is disallowed by default and throws `ExpressionChangedAfterItHasBeenCheckedError`-style loop protection — Angular explicitly guards against effect-induced infinite loops (see Edge Cases).

2.4 Writable vs read-only signals

	<code>signal()</code>	<code>computed()</code>
Mutable	Yes: <code>.set()</code> , <code>.update()</code>	No
Value source	External assignment	Derived from other signals
Evaluation	Eager (value stored directly)	Lazy + memoized
Type	<code>writableSignal<T></code>	<code>Signal<T></code>

`writableSignal<T>` extends `Signal<T>`, so a writable signal can always be passed anywhere a read-only `Signal<T>` is expected — useful for exposing a private mutable signal as a public read-only one:

```
export class CounterService {
  private _count = signal(0);
  readonly count = this._count.asReadonly(); // Signal<number>, no .set()/.update() exposed

  increment() { this._count.update(v => v + 1); }
}
```

`asReadonly()` returns a `Signal<T>` view over the same underlying signal — it's not a copy, so it stays in sync with the original but callers can't mutate through it. This is the standard way to encapsulate mutable state inside a service while giving consumers a safe, read-only handle.

2.5 Signal-based component inputs: `input()` and `input.required()`

Since Angular 17.1/19, component inputs can be declared as signals instead of `@Input()` decorated properties:

```
import { Component, input } from '@angular/core';

@Component({ ... })
export class UserCardComponent {
  // optional input with default value
  name = input('Guest');

  // optional input, no default → type is `string | undefined`
  nickname = input<string>();

  // required input – must be bound by the parent, or Angular throws at compile/runtime
  userId = input.required<string>();

  // input with a transform function (e.g., coercing attribute strings)
  disabled = input(false, { transform: (v: boolean | string) => !!v });

  // aliasing the public input name
  fullName = input('', { alias: 'name' });
}
```

`name` here is a **read-only** `Signal<string>` — read it with `this.name()`, and it updates reactively whenever the parent rebinds the input. You cannot call `.set()` on it — inputs are one-way, parent → child, by design; that's what `model()` is for when two-way binding is needed.

`input.required<T>()` enforces (at compile time, via the Angular compiler/template type checking) that the parent binds a value; omitting the binding is a template type error.

2.6 `model()` — two-way signal binding

`model()` creates an input that also supports being written back to, generating an implicit companion output event so the parent can use Angular's `[(banana-in-a-box)]` syntax:

```
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <button (click)="decrement()">-</button>
    {{ value() }}
    <button (click)="increment()">+</button>
  `,
})
export class CounterComponent {
  value = model(0); // writableSignal-like, plus an implicit `valueChange` output

  increment() { this.value.update(v => v + 1); }
  decrement() { this.value.update(v => v - 1); }
}
```

Parent usage:

```
<app-counter [(value)]="parentCount" />
```

Under the hood, `model()` generates both an input named `value` and an output named `valuechange` (an `EventEmitter`-compatible signal-driven event), which is exactly what `[(value)]` two-way binding syntax expects. `model.required<T>()` exists analogously to `input.required<T>()` for two-way bindings that must be provided.

Unlike `input()`, the child **can write to a `model()`** (`this.value.set(...)` / `.update(...)`) — writes propagate up to the parent's bound property, and the parent's own re-binding propagates down, all through the same signal graph.

2.7 `output()` — signal-era event emitters

Complementing signal inputs, `output()` replaces the decorator-based `@Output() event = new EventEmitter<T>()` pattern with a plain function call:

```
import { Component, output } from '@angular/core';

@Component({ ... })
export class SearchBoxComponent {
  search = output<string>();

  onEnter(value: string) {
    this.search.emit(value);
  }
}
```

`output()` is not itself a signal (it has no `()` read form) — it's a lighter-weight emitter API designed to pair naturally with `input()` / `model()` in signal-based components, and it integrates with `outputFromObservable()` / `outputToObservable()` for RxJS interop.

2.8 Signal-based queries: `viewChild()` / `contentChild()`

Rounding out signal-based component APIs, view/content queries can also be declared as signals rather than decorators:

```
import { Component, viewChild, ElementRef } from '@angular/core';

@Component({ ... })
export class MyComponent {
  inputRef = viewChild<ElementRef<HTMLInputElement>>('searchInput');
  // inputRef is a Signal<ElementRef<HTMLInputElement> | undefined>

  focus() {
    this.inputRef()?.nativeElement.focus();
  }
}
```

`viewChild.required()` mirrors `input.required()` — throws if the queried element/component doesn't exist.

2.9 `untracked()` — reading without subscribing

`untracked()` lets you read a signal's value **inside a reactive context (computed/effect) without registering it as a dependency**:

```
import { signal, effect, untracked } from '@angular/core';

const userId = signal(1);
const debugMode = signal(false);

effect(() => {
  console.log('User changed:', userId()); // tracked – effect re-runs when userId changes
  if (untracked(debugMode)) {             // read, but NOT tracked
    console.log('Debug mode is on');
  }
});
```

Covered in depth in section 5.

3. Code Examples

3.1 A small reactive store pattern with signals

```
import { Injectable, signal, computed } from '@angular/core';

interface Todo {
```

```

    id: number;
    text: string;
    done: boolean;
  }

  @Injectable({ providedIn: 'root' })
  export class TodoStore {
    private todos = signal<Todo[]>([]);

    // public read-only views
    readonly all = this.todos.asReadonly();
    readonly pending = computed(() => this.todos().filter(t => !t.done));
    readonly completedCount = computed(() => this.todos().filter(t => t.done).length);

    add(text: string) {
      this.todos.update(list => [...list, { id: Date.now(), text, done: false }]);
    }

    toggle(id: number) {
      this.todos.update(list =>
        list.map(t => (t.id === id ? { ...t, done: !t.done } : t))
      );
    }
  }
}

```

Note the immutable update pattern: `.update()` receives the current array and must return a *new* array (or object) — mutating the existing array in place (`list.push(...)`) would not trigger change detection reliably, since signals compare by reference/ `object.is` by default.

3.2 `computed()` chaining and dynamic dependencies

```

import { signal, computed } from '@angular/core';

const useMetric = signal(true);
const celsius = signal(20);
const fahrenheit = computed(() => celsius() * 9 / 5 + 32);

// dependency set changes based on which branch runs
const displayTemp = computed(() =>
  useMetric() ? `${celsius()}°C` : `${fahrenheit()}°F`
);

console.log(displayTemp()); // "20°C" — depends on useMetric + celsius
useMetric.set(false);
console.log(displayTemp()); // "68°F" — now depends on useMetric + fahrenheit (which depends on celsius)

```

3.3 `effect()` with cleanup

```

import { Component, signal, effect } from '@angular/core';

```

```

@Component({ ... })
export class PollingComponent {
  intervalMs = signal(1000);

  constructor() {
    effect((onCleanup) => {
      const ms = this.intervalMs();
      const id = setInterval(() => console.log('tick'), ms);

      onCleanup(() => clearInterval(id));
    });
  }
}

```

Every time `intervalMs` changes, the previous interval is cleared (`onCleanup` runs before the next execution, and once more on effect/component destruction) and a new one is scheduled at the updated rate.

3.4 Full signal-based component: inputs, model, output together

```

import { Component, input, model, output, computed } from '@angular/core';

@Component({
  selector: 'app-quantity-picker',
  standalone: true,
  template: `
    <button (click)="decrement()" [disabled]="quantity() <= min()">-</button>
    <span>{{ quantity() }}</span>
    <button (click)="increment()" [disabled]="quantity() >= max()">+</button>
    <small>{{ statusLabel() }}</small>
  `,
})
export class QuantityPickerComponent {
  min = input(0);
  max = input(99);
  quantity = model(1); // two-way bindable
  limitReached = output<'min' | 'max'>();

  statusLabel = computed(() =>
    this.quantity() === this.max() ? 'Max reached' : `_${this.quantity()} selected`
  );

  increment() {
    if (this.quantity() < this.max()) {
      this.quantity.update(v => v + 1);
    } else {
      this.limitReached.emit('max');
    }
  }

  decrement() {
    if (this.quantity() > this.min()) {
      this.quantity.update(v => v - 1);
    }
  }
}

```



```

    } else {
      this.limitReached.emit('min');
    }
  }
}

```

```

<app-quantity-picker [min]="0" [max]="10" [(quantity)]= "cartQuantity"
(limitReached)="onLimit($event)" />

```

4. Internal Working

4.1 The reactive graph: producers, consumers, and dependency tracking

Angular's signal implementation (`@angular/core/primitives/signals`) models a graph of **producers** (signals — including `computed()`) and **consumers** (`computed()`, `effect()`, and Angular's own template bindings).

The mechanism:

1. Every signal has a monotonically increasing **version counter**. `.set()` / `.update()` bumps the version and marks the signal "dirty" relative to any consumer that last saw an older version.
2. When a `computed()` or `effect()` function runs, Angular sets a **global "current consumer"** context before invoking it. Every time a signal is read (`someSignal()`) inside that function, the signal registers itself as a dependency of the current consumer, and the consumer records which signal + which version it last read.
3. After the function finishes running, the tracking context is popped. The consumer now has a precise list of "producers I depend on, and at what version I last saw them."
4. When a producer's value changes, it doesn't eagerly push the new value to every consumer. Instead, it marks itself dirty and **notifies direct consumers that they may be stale** — this is the "push" part. Actual recomputation is **pulled lazily** on next read (for `computed()`) or performed by the scheduler (for `effect()`). This push-then-pull design is what makes the system **glitch-free**: if signal A changes and both B (`computed` depending on A) and C (`computed` depending on both A and B) exist, C is only recomputed once, with fully up-to-date values of both A and B — never with a stale B.

4.2 How `computed()` memoizes

A `computed()` signal caches its last-computed value and the versions of the producers it read to compute it. On read:

- If none of its recorded dependency versions have changed since the last computation, it returns the cached value immediately — no re-execution.
- If any dependency is dirty, it re-runs the derivation function, tracking dependencies fresh (allowing the dependency set to change between runs), stores the new value, and updates its own version counter — but *only if the new value differs from the old one under the equality function*. This means a `computed()` that recomputes to the *same* value (by `object.is` or custom `equal`) does **not** bump its own version, so *its* consumers won't be marked dirty either — memoization and glitch-free propagation compose to prevent unnecessary work from rippling further down the graph.

4.3 How effects are scheduled

`effect()` callbacks are **not run synchronously** when a dependency changes. Instead:

- Angular tracks dirty effects and schedules a flush via the `ChangeDetectionScheduler` (a microtask-based scheduler, since Angular 17-ish), which runs after the current synchronous execution context finishes (similar to how Promise microtasks are flushed) and is coalesced with Angular's change detection tick.
- This means several synchronous signal writes in the same tick (`count.set(1); count.set(2); count.set(3);`) will typically cause the effect to run **once**, seeing the final value — batching, not once-per-write.
- Effects run in the injector context they were created in and are automatically destroyed when that context (component, directive) is destroyed, via `DestroyRef` integration.

4.4 Interaction with change detection and zoneless mode

In a traditional (zone-full) app, template bindings that read signals (`{{ count() }}`) are still ultimately refreshed by Angular's normal change detection pass — but Angular is smarter about it: it marks only the components whose templates read a changed signal as needing a check ("OnPush-like" targeted marking), rather than relying purely on Zone.js's "something async happened, check everything" model.

In **zoneless** mode (`provideExperimentalZonelessChangeDetection()` / `provideZonelessChangeDetection()`), there is no Zone.js patching of `setTimeout`/promises/DOM events at all. Angular instead relies **entirely** on explicit "hey, something might need updating" notifications, and the primary source of those notifications is: signal writes (via the same producer/consumer graph), plus a few other APIs (`ChangeDetectorRef.markForCheck()`, async pipe, host event bindings, attaching views). This is why signals are described as the reactivity primitive that unlocks zoneless Angular — they give Angular a **precise, synchronous notification mechanism** to replace Zone.js's blunt one. In zoneless apps, if you mutate state without going through a signal (or without manually calling `markForCheck()`), Angular has no way to know the UI needs updating — this is a common gotcha when migrating.

5. Edge Cases & Gotchas

5.1 Effects for state derivation is an anti-pattern

Using `effect()` to set another signal based on the first is tempting but wrong:

```
// BAD
const price = signal(100);
const doubled = signal(0);
effect(() => doubled.set(price() * 2)); // works, but...
```

This is strictly worse than `computed()`: it's eager/async instead of lazy, it's a side effect where a pure derivation is what's intended, and it's easy to accidentally create feedback loops. Always prefer `computed()` for "value derived from other signals."

5.2 Writing to a signal read by the same effect → infinite loop protection

```
const count = signal(0);

effect(() => {
  console.log(count());
  count.set(count() + 1); // writing to a signal this effect also reads
});
```

Angular detects this and throws `NG0600: writing to signals is not allowed in a computed or effect by default` (the exact error is something like "effect() cannot write to signals it reads, to avoid infinite loops"). If you genuinely need to write to a signal from inside an effect (e.g., syncing external state), you must opt in with `{ allowSignalWrites: true }` — but even then, if the write feeds back into a signal the effect *also* reads, you can still create a real infinite loop; the flag only lifts the compile-time guard, not the logical hazard.

```
effect(() => {
  console.log(count());
  otherSignal.set(count() * 2); // OK: writes to a signal NOT read by this effect
}, { allowSignalWrites: true });
```

5.3 `untracked()` — reading without creating a dependency

Sometimes you need to read a signal's *current* value inside an effect/computed without wanting changes to that signal to re-trigger the computation:

```
const userId = signal(1);
const analyticsEnabled = signal(true);

effect(() => {
  const id = userId(); // tracked dependency
  if (untracked(() => analyticsEnabled())) {
    logPageView(id);
  }
});
```

Here, toggling `analyticsEnabled` alone will **not** re-run the effect — only changes to `userId` will. This is important for:

- Reading "configuration" signals inside an effect that should only react to "data" signals.
- Avoiding accidental extra dependencies when calling helper functions that happen to read signals internally, inside an effect/computed body.
- Breaking a would-be circular dependency: reading a signal with `untracked()` inside its own effect is a supported way to check the current value without registering the effect as one of its own consumers.

`untracked()` also has an overload that just unwraps a value: `untracked(signal)` is shorthand for `untracked(() => signal())`.

5.4 Object/array mutation doesn't trigger updates

```
const items = signal<string[]>([]);
items().push('new item'); // MUTATES the underlying array in place – no notification fires!
```

Because signal change detection is based on reference/ `Object.is` equality (or a custom `equal`), mutating the object/array *returned by* a signal read does not go through `.set()` / `.update()`, so no version bump occurs and no consumers are notified. Always replace with a new reference:

```
items.update(list => [...list, 'new item']); // correct
```

5.5 `computed()` must be pure — no side effects, no signal writes

`computed()` functions can be re-run at unpredictable times (whenever something reads them after a dependency changed) and possibly re-run multiple times without their result being used (e.g., if nothing ever reads them after a change) — so any side effect placed inside a `computed()` (logging, HTTP calls, writing to another signal) is a bug waiting to happen: it may run more or fewer times than you expect, and Angular does not guarantee the *number* of times a `computed()` re-executes internally, only that reads return a consistent, up-to-date value.

5.6 `input()` outside an injection/field-initializer context

`input()`, `model()`, `output()`, `ViewChild()` etc. must be assigned to class fields at the top level of a component/directive class body (or use the `inject()`-style pattern within a valid context) — they rely on being called during class field initialization so Angular can associate them with the right component definition. Calling `input()` conditionally or inside a method throws at runtime.

5.7 Required inputs/models without a binding

`input.required()` and `model.required()` are enforced by Angular's **template type checker** at compile time if strict templates are enabled, and by a runtime check otherwise (throwing if the value is read before being set, in dev). This is stricter than the old `@Input()` where a missing binding just silently resulted in `undefined` — a genuine improvement in interview terms: "signal-based required inputs give compile-time safety that decorator inputs never had."

5.8 Equality function surprises

If you provide a custom `equal` function that considers two very different values "equal" (e.g., only compares an `id` field), then `.set()` with a value that differs in *other* fields will be silently ignored by that signal — dependents won't be notified even though the object reference changed. This is a common self-inflicted bug when reusing a "same id = same object" equality check for objects whose non-id fields still need to trigger UI updates.

6. Interview Questions & Answers

Q1. What is a signal in Angular, and how do you read/write one?

A signal is a wrapper object around a value that notifies interested consumers when the value changes. You read it by calling it as a function — `mySignal()` — and write to a writable signal via `.set(newValue)` (full replace) or `.update(fn)` (derive next value from current value).

Q2. What's the difference between `signal()` and `computed()`?

`signal()` creates writable, independently-settable state. `computed()` creates a read-only, derived value computed from other signals via a pure function; it has no `.set()` / `.update()`, is evaluated lazily on first read, and is memoized — it only recomputes when one of its tracked dependencies has actually changed.

Interviewer intent: checking whether the candidate understands that `computed()` is not just "a cached function call" but a first-class node in the dependency graph, distinct in mutability and evaluation timing from `signal()`.

Q3. How does Angular know which signals a `computed()` or `effect()` depends on? Do you have to declare dependencies manually?

No manual declaration is needed. Angular tracks dependencies automatically: while executing a `computed()` / `effect()` function, it sets a "current consumer" context, and every signal read (i.e., every function call `someSignal()`) during that execution registers itself as a dependency of that consumer. The dependency set is recomputed fresh on every execution, so it can change dynamically if the function takes different branches.

Q4. Why does Angular disallow writing to a signal from within an `effect()` that reads that same signal by default?

To prevent infinite reactive loops: if an effect both reads and writes the same signal, the write would re-trigger the effect, which writes again, forever. Angular's default guard throws an error (`NG0600`-class) rather than allow this silently. You can opt in with `{ allowsignalwrites: true }` if you need an effect to write to signals — but you must ensure it doesn't create a cycle with the signals it reads.

Q5. What does `untracked()` do, and give a real use case.

`untracked()` reads a signal's value inside a reactive context (`computed()` / `effect()`) without registering it as a dependency of that context. Use case: an effect that logs analytics whenever a `userId` signal changes, but also wants to check a `debugMode` flag signal — you don't want toggling `debugMode` alone to re-trigger the analytics logging, so you read it via `untracked(debugMode)`.

Q6. Why doesn't pushing an item into an array signal's value (`items().push(x)`) update the UI?

Because that mutates the array object in place without going through `.set()` / `.update()` — the signal's internal version counter never increments, `Object.is` comparison never even runs (since no write call happened at all), and no consumers are notified. Signals require replacing the value with a new reference: `items.update(list => [...list, x])`.

Q7. What is the difference between `input()` and `model()`?

`input()` creates a one-way, parent-to-child read-only signal input — the child can read it but never write to it. `model()` creates a two-way-bindable input: the child can both read *and* write to it (`.set()` / `.update()`), and Angular auto-generates a companion `xchange` output so the parent can use `[(x)]` box syntax. Use `input()` for pure top-down data; use `model()` when the child needs to mutate state that the parent also owns (e.g., a custom form control or stepper).

Q8. What happens if a parent doesn't bind a value to `input.required<T>()`?

With Angular's strict template type checking enabled, it's a compile-time template error — the build fails. Without strict templates, or if bypassed, it throws at runtime when the input is read before being set. This is stricter than the old `@Input()` decorator, where a missing binding silently produced `undefined` with no error at all.

Interviewer intent: tests whether the candidate knows signal inputs improved on a real, previously-silent footgun in Angular's API — a good "why is this better" talking point.

Q9. Is `effect()` synchronous — does it run immediately when a dependency signal changes?

No. Effects are scheduled asynchronously by Angular's change-detection scheduler and flushed in a microtask-like pass, coalesced with change detection. If you make several synchronous writes to a dependency in the same execution turn, the effect typically runs once afterward, seeing only the final value — not once per write. This batching is intentional, both for performance and to avoid effects observing transient/inconsistent intermediate states.

Q10. Explain "glitch-free" propagation in the context of signals. Why does it matter?

Glitch-free means that when a signal changes, every downstream computation that depends on it (directly or transitively) is guaranteed to see a fully consistent, up-to-date view of the whole graph — never a partially-updated intermediate state. Concretely: if `computed C` depends on both `signal A` and `computed B` (which itself depends on `A`), then when `A` changes, `C` recomputes exactly once, using the *already-updated* `B`, never a stale `B` computed from the old `A`. This is achieved via the push-dirty/pull-recompute design: producers mark consumers dirty eagerly (push) but consumers only actually recompute lazily on next read (pull), after all upstream changes for that "tick" have already been marked. It matters because without it, UIs and derived state could momentarily show or compute internally-inconsistent combinations of old and new values.

Interviewer intent: distinguishes candidates who've only used the signal API from those who understand the reactive graph model well enough to reason about correctness in complex dependency chains.

Q11. How do signals relate to zoneless change detection?

Zoneless Angular removes Zone.js, which previously was the mechanism telling Angular "something async happened, re-check the tree." Without Zone.js, Angular needs another precise way to know when to re-render, and signals provide exactly that: a signal write notifies its dependent consumers (templates, computed signals, effects) directly through the producer/consumer graph, with no reliance on monkey-patched global APIs. This is why signals are considered foundational infrastructure for zoneless mode, not just a nicer state API — in a zoneless app, state changes made outside the signal graph (e.g., mutating a plain class property) will not trigger any UI update unless you manually call `markForCheck()` / `ChangeDetectorRef` methods.

Q12. Why should `computed()` functions be pure, and what can go wrong if they aren't?

`computed()`'s execution timing and frequency are an implementation detail — Angular may run it lazily, may skip re-running it if nothing reads it after a dependency changes, and may in principle re-run it more than once in some scenarios. If the function has side effects (writing another signal, calling an API, mutating external state), those side effects' timing and count become unpredictable and untestable. Side effects belong in `effect()`, which has an explicit, documented execution/scheduling contract; pure derivation belongs in `computed()`.

Q13. How would you expose mutable internal state from a service as read-only to consumers, using signals?

Keep a private `writableSignal` and expose a public read-only view via `.asReadonly()`:

```
private _count = signal(0);
readonly count = this._count.asReadonly();
```

`asReadonly()` returns a `Signal<T>` (no `.set()` / `.update()`) backed by the same underlying state — not a copy — so external code can read and react to it but can only mutate it through the service's own methods (e.g., `increment()`), preserving encapsulation.

Q14. Can a `computed()` signal have a dynamically-changing set of dependencies? Give an example.

Yes — because dependency tracking happens fresh on every execution of the computed function, if the function takes different branches on different runs (e.g., an `if`/ternary that reads different signals depending on another signal's value), the tracked dependency set updates to reflect whichever signals were actually read on the *most recent* run. Example: `computed(() => useMetric() ? celsius() : fahrenheit())` — while `useMetric()` is true, changes to `fahrenheit` won't trigger recomputation (it wasn't read on the last run); flip `useMetric` and the dependency set shifts to include `fahrenheit` instead of `celsius`.

7. Quick Revision Cheat Sheet

- `signal(initial)` → writable, `object.is` (or custom `equal`) comparison, read via `s()`, write via `s.set()` / `s.update()`.
- `computed(fn)` → read-only, lazy, memoized, dependencies auto-tracked and can change per run; keep it pure (no side effects, no writes).
- `effect(fn)` → side effects only; runs async/batched after signal changes; auto-cleanup on destroy; supports `onCleanup`; writing to a signal it also reads throws unless `{ allowsSignalWrites: true }` (and even then, avoid true cycles).
- `untracked(fn | signal)` → read a signal inside a reactive context without registering it as a dependency.
- `input(default)` / `input.required<T>()` → one-way, read-only signal input from parent; `.required` enforced by template type checker/runtime; supports `transform` and `alias`.
- `model(default)` / `model.required<T>()` → two-way signal binding; child can read *and* write; auto-generates `xchange` output for `[(x)]` syntax.
- `output<T>()` → function-based replacement for `@Output() = new EventEmitter(); .emit(value)`.
- `viewChild()` / `contentChild()` → signal-based query alternatives to `@ViewChild` / `@ContentChild`; `.required()` variant throws if not found.
- `asReadonly()` → expose a `writableSignal` as a `Signal` (same underlying state, no external mutation).
- **Mutation gotcha:** never mutate objects/arrays returned by a signal in place — always `.set()` / `.update()` with a new reference.
- **Glitch-free graph:** producers push "dirty" notifications; consumers pull/recompute lazily on next read — guarantees consistent values across the whole dependency graph, no stale intermediate states.
- **Zoneless Angular** relies on the signal graph (plus `markForCheck()`, async pipe, etc.) as its change-notification source, replacing Zone.js's global async patching.